



# CodeSmile

## Reverse Engineering

|                     |                                                   |
|---------------------|---------------------------------------------------|
| <b>RIFERIMENTO</b>  | ReverseEngineering                                |
| <b>VERSIONE</b>     | 0.8                                               |
| <b>DESTINATARIO</b> | Prof. Andrea De Lucia<br>Dott. Giammaria Giordano |



*Laurea Magistrale in Software Engineering & IT Management –Università di Salerno*  
*Corso di Ingegneria del Software: tecniche avanzate*  
*Prof. Andrea De Lucia, Dott. Giammaria Giordano*

## Gestione Gruppo

---

| MEMBRO           | EMAIL                          | MATRICOLA  |
|------------------|--------------------------------|------------|
| MARTINA DE LUCIA | m.delucia18@studenti.unisa.it  | NF22500053 |
| ALESSANDRA RAIA  | a.raia7@studenti.unisa.it      | NF22500054 |
| CARLA STEFANILE  | c.stefanile1@studenti.unisa.it | NF22500097 |



## Version

---

| DATA       | VERSIONE | DESCRIZIONE                          | AUTORI      |
|------------|----------|--------------------------------------|-------------|
| 13/01/2026 | 0.1      | Prima Stesura                        | MDL, AR, CS |
| 14/01/2026 | 0.2      | Descrizione del Sistema              | MDL         |
| 14/01/2026 | 0.3      | Prima Revisione                      | CS          |
| 14/01/2026 | 0.4      | Funzionamento e flussi di esecuzione | AR          |
| 15/01/2026 | 0.5      | Struttura e architettura del sistema | MDL         |
| 15/01/2026 | 0.6      | Seconda Revisione                    | MDL, CS     |
| 15/01/2026 | 0.7      | Terza Revisione                      | AR, CS      |
| 15/02/2026 | 0.8      | Quarta Revisione                     | MDL         |



## Sommario

|                                                                           |    |
|---------------------------------------------------------------------------|----|
| Gestione Gruppo.....                                                      | 2  |
| Version.....                                                              | 3  |
| 1 Descrizione del sistema: CodeSmile .....                                | 5  |
| 1.1 Contesto e motivazione .....                                          | 5  |
| 1.2 Scopo del tool .....                                                  | 5  |
| 1.3 Visione d'insieme e perimetro del repository.....                     | 5  |
| 1.4 Funzionalità principali.....                                          | 6  |
| 1.5 Input, output e formato dei risultati .....                           | 7  |
| 1.6 Componente AI-based: ruolo e relazione con la detection statica ..... | 7  |
| 2 Struttura e Architettura del Sistema.....                               | 8  |
| 2.1 Perimetro .....                                                       | 8  |
| 2.2 Scomposizione in package .....                                        | 9  |
| 2.2.1 cli .....                                                           | 10 |
| 2.2.2 code_extractor .....                                                | 10 |
| 2.2.3 components.....                                                     | 11 |
| 2.2.4 detection_rules .....                                               | 12 |
| 2.2.5 gui .....                                                           | 14 |
| 2.2.6 report .....                                                        | 15 |
| 2.2.7 utils.....                                                          | 16 |
| 2.2.8 data_preparation .....                                              | 16 |
| 2.2.9 finetuning.....                                                     | 18 |
| 2.2.10 webapp .....                                                       | 18 |
| 3 Funzionamento e flussi di esecuzione .....                              | 21 |
| 3.1 Utilizzo tramite Linea di Comando (CLI) .....                         | 21 |
| 3.1.1 Flusso di Utilizzo CLI .....                                        | 21 |
| 3.2 Utilizzo tramite Interfaccia Utente Grafica (GUI) .....               | 23 |
| 3.2.1 Flusso di Utilizzo GUI.....                                         | 23 |
| 3.2 Utilizzo tramite Web App.....                                         | 26 |
| 3.3.1 Flusso di Utilizzo WebApp .....                                     | 26 |



# 1 Descrizione del sistema: CodeSmile

---

## 1.1 Contesto e motivazione

I sistemi software che incorporano componenti di Machine Learning (*ML-enabled systems*) presentano criticità qualitative specifiche rispetto ai sistemi tradizionali. In tali contesti, l'implementazione ruota attorno a pipeline di preparazione dati, addestramento e valutazione dei modelli, con un uso intensivo di librerie specialistiche (ad es. Pandas/NumPy per il data processing, PyTorch/TensorFlow per il training). Questo scenario favorisce l'insorgenza di **Machine Learning-specific code smells (ML-CS)**: pratiche di implementazione subottimali che, pur essendo sintatticamente corrette, possono degradare prestazioni, robustezza, consumo di memoria e manutenibilità della pipeline.

Il lavoro scientifico su CodeSmile parte dal catalogo di 22 ML-CS e osserva che solo un sottoinsieme risulta **efficacemente rilevabile tramite sola analisi statica**. Di conseguenza, CodeSmile è progettato per individuare tale sottoinsieme mediante una combinazione di analisi strutturale (AST) e regole deterministiche.

## 1.2 Scopo del tool

**CodeSmile** è un tool in Python per la **rilevazione automatica** di ML-CS su codice Python mediante **analisi statica**, con produzione di un report strutturato (CSV).

Nel paper, CodeSmile è presentato come uno strumento di analisi statica basato su **AST (Abstract Syntax Tree)** e regole (*rule-based conditions*), progettato per l'identificazione di **12 ML-CS staticamente rilevabili**, formalizzati a partire dalla letteratura. In sintesi, lo scopo operativo del sistema è:

- acquisire in input uno o più progetti Python (o porzioni di essi);
- analizzare il codice staticamente (senza esecuzione);
- segnalare le occorrenze di smell restituendole in un formato consultabile e processabile (CSV).

## 1.3 Visione d'insieme e perimetro del repository

Il repository *smell\_ai* include più modalità e sottosistemi. Ai fini del progetto, è utile distinguere:

### 1. Core di analisi statica (AST + rule based)

È il componente principale: implementa la scansione dei sorgenti, il parsing AST, l'estrazione del contesto e l'applicazione dei rilevatori di smell, producendo report aggregati.

### 2. Componente AI-based (sperimentale/alternativa)

Include una pipeline che esplora l'uso di LLM per generare dataset sintetici e per addestrare un modello fine-tuned dedicato alla classificazione di smell. Questa componente non è necessaria per l'esecuzione della pipeline statica, ma rappresenta un filone di estensione coerente con l'evoluzione del progetto.



### 3. Interfacce di fruizione (CLI/GUI/Web)

Il repository supporta diverse modalità di utilizzo dell'applicazione. In particolare, sono disponibili: CLI (Command Line Interface); GUI (Graphical User Interface); Interfaccia Web.

## 1.4 Funzionalità principali

Le funzionalità del sistema possono essere ricondotte alle seguenti capacità:

### (F1) Analisi statica del sorgente Python tramite AST

Il sistema effettua il parsing dei file e costruisce l'AST, rendendo possibile un'analisi strutturata di chiamate a funzioni, assegnamenti, accessi ad attributi ed operazioni di indexing.

### (F2) Rilevazione di ML-CS tramite regole deterministiche (rule-based)

Ogni ML-CS è implementato tramite una regola specifica che combina:

- pattern strutturali (forma dei nodi AST, chiamate a metodi, uso di parametri)
- vincoli di contesto (es. verificare che l'oggetto coinvolto sia effettivamente un DataFrame o un tensore).

L'approccio rule-based consente di ottenere risultati ripetibili e verificabili, riducendo l'ambiguità tipica delle analisi puramente testuali.

### (F3) Estrazione del contesto (library association, variabili e inferenza di tipo)

Per supportare la detection, il sistema costruisce informazioni di contesto a partire dal sorgente e mantiene una struttura assimilabile a una **symbol table**, cioè una mappatura incrementale che associa identificatori del programma a proprietà rilevanti per l'analisi. In particolare, il contesto include:

- **Librerie importate e relativi alias**, per riconoscere l'appartenenza di chiamate e costrutti a framework specifici (es. identificare che pd si riferisce a Pandas).
- **Variabili definite e modalità di inizializzazione/assegnamento**, tracciando gli identificatori introdotti nella funzione e le principali trasformazioni subite.
- **Associazioni variabile → categoria/ruolo**, ad esempio variabili che rappresentano strutture dati caratteristiche (DataFrame) o oggetti di training/modello/tensori.
- **Segnali e operazioni caratteristiche**, cioè, chiamate e pattern ricorrenti su tali strutture (metodi di DataFrame, operazioni tensoriali, costruttori e API di modello).

Queste informazioni vengono utilizzate dai rilevatori di smell per rendere la detection più precisa, riducendo segnalazioni su oggetti non pertinenti e permettendo verifiche che richiedono contesto (ad esempio distinguere un'operazione generica da un'operazione su DataFrame o tensori).

### (F4) Produzione di report in formato strutturato

L'esito dell'analisi è esportato in CSV, consentendo consultazione, confronto tra esecuzioni e possibili elaborazioni statistiche.



### **(F5) Estendibilità del catalogo di smells**

A livello architetturale, le regole sono organizzate secondo una gerarchia che fa capo a una classe base astratta (Smell) con un'interfaccia uniforme. In particolare, ciascuna regola espone un metodo di detection che restituisce una lista di occorrenze normalizzate con metadati (nome smell, riga, descrizione, informazioni aggiuntive). Questa scelta rende il sistema naturalmente estendibile: l'aggiunta di un nuovo smell consiste nell'introdurre una nuova regola compatibile con l'interfaccia e registrarla nel flusso di analisi.

## **1.5 Input, output e formato dei risultati**

**Input.** Il caso d'uso primario prevede l'analisi di una directory contenente file Python (progetto singolo o batch di progetti).

**Output.** Il risultato è un report CSV che, per ogni occorrenza di smell, conserva informazioni minime per la tracciabilità: file coinvolto, contesto (tipicamente funzione/unità analizzata), identificativo dello smell e metadati (ad es. riga e note descrittive). L'implementazione delle regole è coerente con l'idea di restituire occorrenze come record strutturati con campi descrittivi e "additional info".

## **1.6 Componente AI-based: ruolo e relazione con la detection statica**

Accanto al core statico, il repository include una pipeline AI-based descritta come studio sperimentale di fattibilità sull'uso di LLM per:

- generazione di dati sintetici mediante "iniezione" di smell in funzioni;
- addestramento (fine-tuning, LoRA) di un modello per classificazione degli smells;
- inferenza con configurazioni ottimizzate (quantizzazione, esecuzione locale).

Dal punto di vista del progetto, questa componente è rilevante perché:

- dimostra una direzione evolutiva coerente con l'estensione proposta (integrare LLM per nuove tipologie di smell o per modalità di detection più flessibili);
- introduce la possibilità di affiancare alla detection deterministica un approccio "probabilistico" basato su modelli, potenzialmente utile per smell non intercettabili staticamente o per casi in cui serva maggiore comprensione semantica del codice.



## 2 Struttura e Architettura del Sistema

---

L'analisi architeturale mira a:

- identificare i **confini del sistema** e le sue principali funzionalità operative;
- descrivere la **decomposizione modulare** del repository in sottosistemi coerenti, evidenziando dipendenze e relazioni;
- ricostruire i componenti del core runtime e la loro collaborazione lungo il percorso di esecuzione principale;

### 2.1 Perimetro

Il repository *smell\_ai* include più componenti (analisi statica, sottosistema AI-based e diverse interfacce di fruizione). Ai fini del reverse engineering, questo documento adotta un perimetro articolato su più livelli, in modo da mantenere il focus sul comportamento principale del sistema senza trascurare i sottosistemi presenti nel repository.

#### **Perimetro principale (GUI e core runtime statico).**

L'analisi si concentra in via prioritaria sulla pipeline di analisi statica e sulla relativa orchestrazione tramite GUI, poiché:

- rappresentano il flusso principale di esecuzione rilevante per il processo di manutenzione;
- producono gli artefatti di output più significativi per tracciabilità e regressione (report CSV);
- costituiscono la base architeturale su cui si innestano le evoluzioni richieste dal progetto (configurazione, avvio e controllo dell'analisi).

Rientrano nel perimetro principale:

- orchestrazione dell'esecuzione lato interfaccia (GUI: input utente, validazioni, avvio/annullamento, stato e log);
- orchestrazione e analisi progetto (core: ProjectAnalyzer e componenti di coordinamento);
- ispezione del sorgente (parsing AST e unità di analisi);
- estrazione del contesto (extractors e strutture dati di supporto);
- applicazione delle regole (catalogo dei rilevatori e infrastruttura di rule checking);
- generazione e aggregazione del report.

#### **Perimetro esteso (interfacce e sottosistemi presenti nel repository).**

Il documento include inoltre una vista architeturale dei seguenti elementi, considerati come sottosistemi complementari:





- **CLI:** interfaccia a riga di comando che consente l'esecuzione della pipeline senza ricorrere alla GUI. È trattata come thin layer alternativo che configura e invoca il core runtime.
- **Pipeline AI-based:** componenti per generazione di dataset, fine-tuning e inferenza tramite LLM. Essa è considerata come sottosistema separato ma parte integrante del repository; l'analisi ne ricostruisce i componenti principali e le dipendenze, al fine di chiarirne il ruolo e l'eventuale riusabilità nelle evoluzioni basate su LLM.

### **Fuori perimetro (solo contestualizzazione).**

La Web Application e i microservizi, ove presenti, sono descritti sinteticamente come ulteriore canale di fruizione. Non costituiscono tuttavia l'oggetto principale del reverse engineering di questa fase, poiché introducono aspetti di deployment e integrazione (gateway, servizi, comunicazione HTTP) che non modificano la logica interna del core runtime e che richiederebbero una trattazione specifica separata.

## **2.2 Scomposizione in package**

La struttura del repository CodeSmile è organizzata in package che riflettono una separazione delle responsabilità secondo macro-aree funzionali. La vista a package è utilizzata in questa fase per:

1. individuare i principali sottosistemi del sistema,
2. evidenziare dipendenze e punti di interazione tra moduli,
3. identificare i confini del core runtime rispetto a componenti accessorie (interfacce, pipeline AI-based).

La decomposizione in package risponde a esigenze di modularità e manutenibilità: ciascun package raggruppa elementi coesi per responsabilità (orchestrazione, analisi, estrazione contesto, regole, reportistica), riducendo l'accoppiamento tra parti del sistema e rendendo più semplice l'evoluzione incrementale tramite l'aggiunta o la sostituzione di componenti.

In particolare, nella vista package adottata in questo documento, i moduli vengono aggregati in tre insiemi principali:

1. **Core runtime (analisi statica):** include i package responsabili dell'esecuzione della pipeline di detection (scansione sorgenti, parsing AST, estrazione del contesto, applicazione delle regole e produzione del report). Questo insieme rappresenta il cuore operativo del sistema e costituisce il riferimento primario per i diagrammi e per la ricostruzione dei flussi.
2. **Presentation layer (interfacce di fruizione):** include i package che forniscono modalità alternative di avvio e controllo dell'analisi (ad esempio CLI e GUI). Tali componenti non implementano la detection, ma configurano e invocano il core runtime, gestendo input utente e visualizzazione/log dei risultati.
3. **Sottosistemi complementari:** includono componenti separati dal core runtime, come la pipeline AI-based e (ove presenti) i moduli della Web Application e dei servizi. Questi sottosistemi sono inclusi nella vista per completezza e per chiarire



le relazioni con il nucleo statico, ma vengono trattati con un livello di dettaglio proporzionato al loro ruolo nel perimetro del documento.

### 2.2.1 cli

**Scopo:** Fornisce l'interfaccia a riga di comando per avviare e configurare l'analisi statica di progetti Python alla ricerca di code smell specifici del Machine Learning.

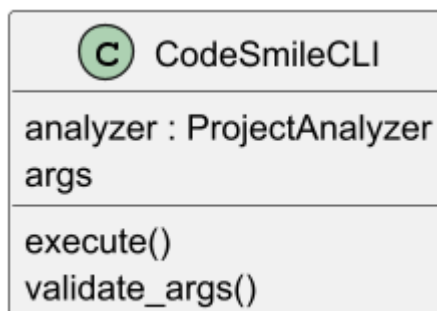
#### Modulo principale:

- **cli\_runner.py:** Definisce la classe CodeSmileCLI che gestisce l'interfaccia a linea di comando. Si occupa del parsing degli argomenti tramite `argparse`, della validazione dei parametri di input/output, e dell'orchestrazione dell'analisi delegando a `ProjectAnalyzer`. Supporta esecuzione sequenziale e parallela (multi-processo), modalità multi-progetto per batch analysis, e ripresa da checkpoint. Espone la funzione `main()` come entry point dell'applicazione. I risultati vengono salvati in formato CSV.

#### Parametri CLI supportati:

- `--input`: path al progetto o directory di progetti da analizzare (*obbligatorio*)
- `--output`: path di output per i risultati CSV (*obbligatorio*)
- `--parallel`: abilita esecuzione parallela multi-processo (*opzionale*)
- `--max_workers`: numero di worker per parallelizzazione (*opzionale*)
- `--resume`: riprende analisi interrotta da checkpoint (*opzionale*)
- `--multiple`: attiva modalità batch per analisi di più progetti (*opzionale*)

**Integrazione:** Il package cli funge da thin layer tra l'utente e i componenti core (`ProjectAnalyzer`, `Inspector`, `RuleChecker`), traducendo i comandi CLI in chiamate appropriate senza contenere logiche di analisi complesse.



### 2.2.2 code\_extractor

**Scopo:** Raccoglie moduli specializzati nell'estrazione di strutture semantiche, contesto e costrutti tipici delle librerie ML tramite analisi AST (Abstract Syntax Tree), fornendo i dati necessari ai rilevatori di smell per l'analisi statica.

#### Moduli principali:

- **library\_extractor.py:** Rileva librerie importate nel codice e i relativi alias. Analizza statement di `import` (`import X as Y`) e `from-import` (`from X import Y`)



tramite AST, mappando chiamate a funzioni o metodi alla libreria di provenienza. Fornisce dizionari alias-libreria utilizzati dagli altri extractor.

- **model\_extractor.py**: Gestisce informazioni su modelli ML e operazioni su tensori. Carica dizionari di modelli e operazioni tensoriali da file CSV in `obj_dictionaries` (`models.csv`, `tensors.csv`), filtra operazioni multi-tensore e verifica l'appartenenza di metodi a librerie note (TensorFlow, PyTorch, Keras, scikit-learn). Utilizzato per smell specifici di framework ML.
- **dataframe\_extractor.py**: Identifica variabili DataFrame Pandas, metodi richiamati e colonne accessibili tramite analisi AST. Utilizza un dizionario di metodi Pandas caricato da CSV (`dataframes.csv`) per affinare il rilevamento e tracciare accessi a colonne e operazioni dataframe-specific.
- **variable\_extractor.py**: Estrae definizioni e utilizzi delle variabili dal codice, collegandole ai nodi AST. Traccia assegnazioni, scope e data flow per supportare l'analisi statica. Fondamentale per smell che richiedono tracking di stato (es. memory leaks, variable misuse).

**Integrazione:** Tutti i moduli vengono istanziati e orchestrati dalla classe `Inspector`, che li utilizza in sequenza per costruire un contesto di analisi completo (librerie, modelli, dataframe, variabili) da passare al `RuleChecker` per la detection degli smell.

| DataFrameExtractor                                                                                             |
|----------------------------------------------------------------------------------------------------------------|
| <code>df_methods : list</code>                                                                                 |
| <code>extract_dataframe_variables(fun_node: ast.AST, alias: str) -&gt; list[str]</code>                        |
| <code>load_dataframe_dict(path: str)</code>                                                                    |
| <code>track_dataframe_accesses(fun_node: ast.AST, dataframe_vars: list[str]) -&gt; dict[str, list[str]]</code> |
| <code>track_dataframe_methods(fun_node: ast.AST, dataframe_vars: list[str]) -&gt; dict[str, list[str]]</code>  |

| LibraryExtractor                                                                       |
|----------------------------------------------------------------------------------------|
| <code>extract_libraries(tree: ast.AST) -&gt; list[dict[str, str]]</code>               |
| <code>get_library_aliases(libraries: list[dict[str, str]]) -&gt; dict[str, str]</code> |
| <code>get_library_of_node(node: ast.AST, aliases: dict[str, str]) -&gt; str</code>     |

| ModelExtractor                                                               |
|------------------------------------------------------------------------------|
| <code>model_dict : NoneType</code>                                           |
| <code>models_path : str</code>                                               |
| <code>tensor_operations_dict : NoneType</code>                               |
| <code>tensors_path : str</code>                                              |
| <code>check_model_method(model: str, libraries: list[str]) -&gt; bool</code> |
| <code>load_model_dict() -&gt; dict[str, list]</code>                         |
| <code>load_model_methods() -&gt; list[str]</code>                            |
| <code>load_tensor_operations_dict() -&gt; dict[str, list]</code>             |

| VariableExtractor                                                                        |
|------------------------------------------------------------------------------------------|
| <code>extract_variable_definitions(fun_node: ast.AST) -&gt; dict[str, ast.Assign]</code> |
| <code>track_variable_usage(fun_node: ast.AST) -&gt; dict[str, list[ast.Name]]</code>     |

### 2.2.3 components

**Scopo:** Contiene i componenti core per l'orchestrazione dell'analisi: coordinamento degli estrattori, applicazione delle regole di detection e gestione dell'analisi a livello di file e progetto.

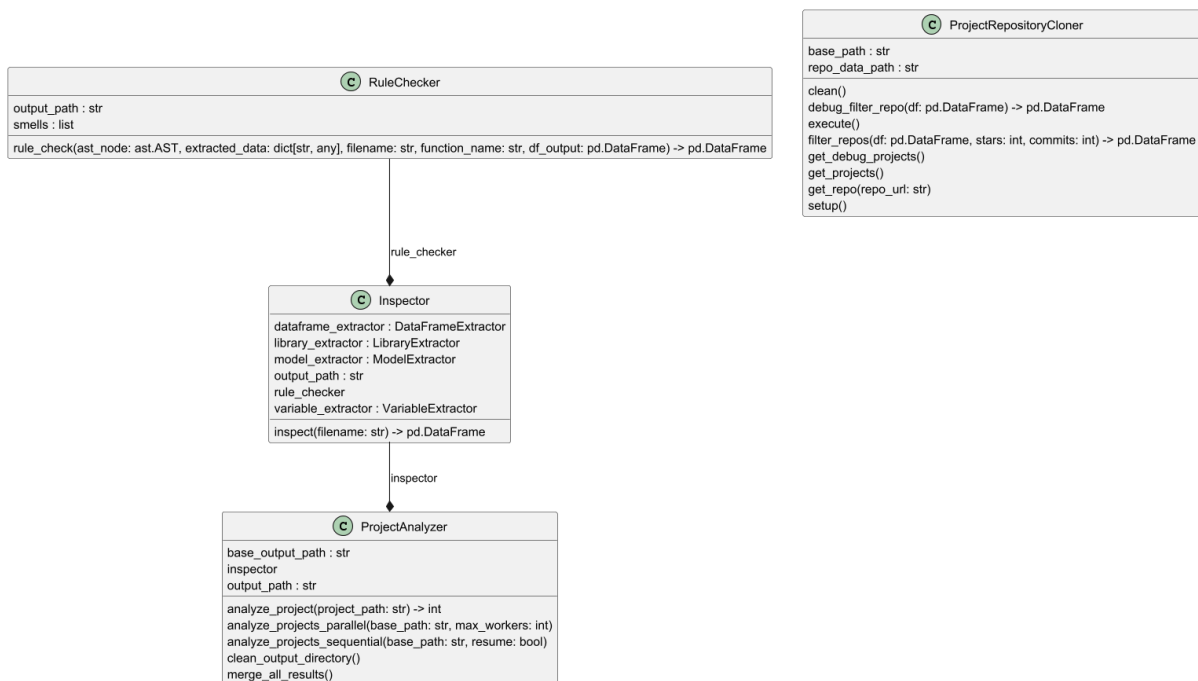
#### Moduli principali:

- **inspector.py**: Definisce la classe `Inspector`, responsabile dell'analisi di singoli file Python. Costruisce l'AST tramite `ast.parse()`, coordina in sequenza i quattro estrattori semantici (`LibraryExtractor`, `ModelExtractor`, `DataFrameExtractor`, `VariableExtractor`) per raccogliere il contesto di analisi, quindi delega a `RuleChecker` l'applicazione delle regole. Restituisce un `DataFrame Pandas` contenente gli smell rilevati per il file analizzato.



- **rule\_checker.py:** Definisce la classe RuleChecker che applica le regole di detection sull'AST. Carica dinamicamente tutte le classi smell da detection\_rules (6 API-specific + 10 generic), invoca il metodo detect() di ciascuna regola passando nodo AST e dati estratti, aggrega i risultati in un DataFrame strutturato.
- **project\_analyzer.py:** Definisce la classe ProjectAnalyzer che gestisce l'analisi di interi progetti. Naviga ricorsivamente la struttura del progetto identificando file Python, orchestra l'esecuzione tramite Inspector in modalità sequenziale o parallela (con ThreadPoolExecutor), implementa checkpoint per resume, gestisce progresso e logging, delega la generazione dei report finali al modulo report.
- **project\_repository\_cloner.py:** Definisce la classe ProjectRepositoryCloner per clonare e gestire repository dal dataset ML. Supporta filtri per metriche (stars, commits, LOC), si occupa del setup (clone) e cleanup (rimozione) dei progetti per analisi batch, utilizzato principalmente nelle pipeline di data preparation e validazione.

**Integrazione:** ProjectAnalyzer utilizza Inspector, che a sua volta coordina RuleChecker e i quattro estrattori. Questo package rappresenta il cuore orchestrativo dell'intero sistema di analisi statica.



## 2.2.4 detection\_rules

**Scopo:** Definisce la gerarchia delle regole di detection per identificare code smell specifici di librerie ML e pattern generici tramite analisi AST. Implementa il pattern Strategy con una classe base astratta e 16 rilevatori concreti organizzati in due sotto-package.

**Modulo base:**



- **smell.py**: Classe astratta Smell che definisce l'interfaccia comune per tutti i rilevatori. Ogni sottoclasse implementa `detect(ast_node, extracted_data)` che restituisce una lista di dizionari con chiavi: `name`, `line`, `description`, `additional_info`. Fornisce anche `format_smell()` per la formattazione standardizzata dei risultati.

#### Sotto-package **api\_specific**/ (6 smell specifici di API ML):

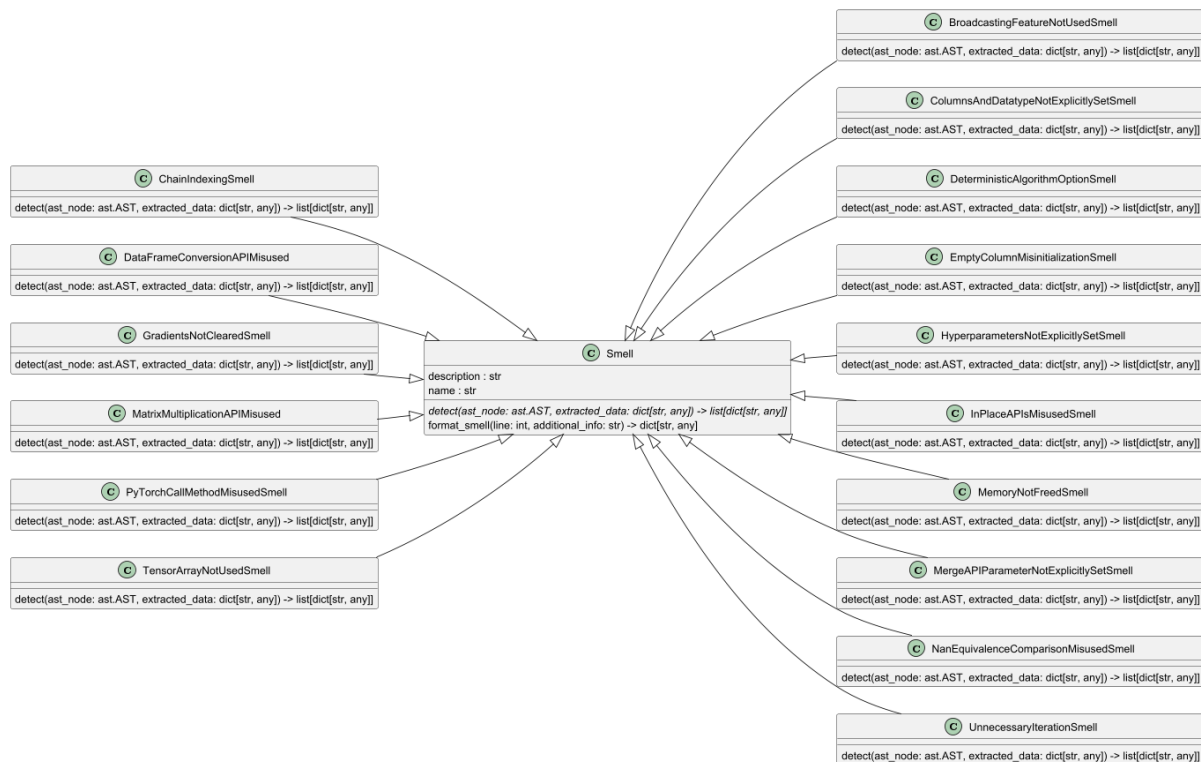
- **chain\_indexing\_smell.py**: Rileva chained indexing su DataFrame Pandas (`df['col'][0]` invece di `df.loc[0, 'col']`).
- **dataframe\_conversion\_api\_misused.py**: Segnala uso di `values` deprecato, suggerendo `to_numpy()`.
- **gradients\_not\_cleared\_before\_backward\_propagation.py**: Verifica che `optimizer.zero_grad()` preceda `loss.backward()` nei training loop PyTorch.
- **matrix\_multiplication\_api\_misused.py**: Sconsiglia `np.dot()` per moltiplicazioni matriciali, suggerendo `np.matmul()` o `@`.
- **pytorch\_call\_method\_misused.py**: Intercetta chiamate dirette a `forward()`, suggerendo invocazione diretta (`model(x)` invece di `model.forward(x)`).
- **tensor\_array\_not\_used.py**: Suggerisce `tf.TensorArray()` quando `tf.constant()` viene aggiornato in loop con `tf.concat()`.

#### Sotto-package **generic**/ (10 smell generici):

- **broadcasting\_feature\_not\_used.py**: Rileva uso inefficiente di `tf.tile()` invece del broadcasting automatico.
- **columns\_and\_datatype\_not\_explicitly\_set.py**: Richiede specifica esplicita di `dtype` e `columns` per DataFrame o `pd.read_csv()`.
- **deterministic\_algorithm\_option\_not\_used.py**: Avvisa su `torch.use_deterministic_algorithms(True)` per impatto prestazioni.
- **empty\_column\_misinitialization.py**: Vieta inizializzazione colonne con `0` o `""`, consigliando `pd.NA` o `np.nan`.
- **hyperparameters\_not\_explicitly\_set.py**: Segnala modelli ML istanziati senza iperparametri espliciti.
- **in\_place\_apis\_misused.py**: Verifica uso coerente del parametro `inplace` nelle API Pandas.
- **memory\_not\_freed.py**: Impone di `torch.cuda.empty_cache()` per modelli in loop.
- **merge\_api\_parameter\_not\_explicitly\_set.py**: Richiede parametri espliciti `how`, `on`, `validate` in `merge()` di Pandas, evitando comportamenti anomali.
- **nan\_equivalence\_comparison\_misused.py**: Evita confronti diretti con `np.nan`, suggerendo `pd.isna()` o `np.isnan()`.
- **unnecessary\_iteration.py**: Rileva iterazioni inefficienti su DataFrame (`iterrows`, `apply` con lambda semplici, `applymap`).



**Integrazione:** Tutti i rilevatori vengono caricati dinamicamente da RuleChecker tramite reflection, applicati in sequenza ai nodi AST con i dati estratti, implementando una pipeline di detection modulare ed estendibile.



## 2.2.5 gui

**Scopo:** Fornisce un'interfaccia grafica basata su Tkinter per configurare ed eseguire l'analisi statica con visualizzazione in tempo reale dei log di esecuzione.

**Moduli principali:**

- **code\_smell\_detector\_gui.py:** Definisce la classe CodeSmellDetectorGUI che costruisce l'interfaccia utente completa. Gestisce widget Tkinter per la raccolta di percorsi input/output, checkbox per modalità parallela/resume/multiple, spinner per numero di worker. Lancia l'analisi su un thread dedicato tramite ProjectAnalyzer per evitare il blocco dell'UI. Include gestione eventi per bottoni, validazione input e aggiornamento dinamico dell'interfaccia durante l'analisi.
- **gui\_runner.py:** Script entry point che istanzia Tk() root window, crea CodeSmellDetectorGUI e avvia il mainloop Tkinter. Rappresenta il punto di ingresso per l'esecuzione in modalità grafica (python -m gui.gui\_runner).
- **textbox\_redirect.py:** Implementa un writer custom (io.StringIO-like) che intercetta e redirige stdout nel widget Text della GUI. Permette la visualizzazione in tempo reale dei log generati da ProjectAnalyzer durante l'analisi, migliorando la user experience con feedback progressivo.

**Integrazione:** La GUI utilizza ProjectAnalyzer per l'analisi, operando in modalità asincrona tramite threading per mantenere l'interfaccia responsive. Rappresenta



un'alternativa user-friendly al package cli per utenti meno esperti o per scenari di analisi interattiva.



## 2.2.6 report

**Scopo:** Generazione di report aggregati, analisi statistiche e visualizzazioni sugli smell rilevati dall'analisi statica.

### Modulo principale:

- **report\_generator.py:** Classe `ReportGenerator` che post-elabora i risultati CSV. Carica e fa merge multipli CSV tramite `_load_data()`, genera report per tipo smell (`smell_report()`), per progetto (`project_report()`), workbook Excel multi-sheet (`summary_report()`), e visualizzazioni grafiche (`visualize_smell_report()`). Include interfaccia CLI interattiva tramite `menu()` e metodo orchestratore `run()` per la gestione dell'I/O.

**Integrazione:** Utilizzato da `ProjectAnalyzer` al termine dell'analisi per produrre report riassuntivi. Può essere invocato indipendentemente come post-processing tool su risultati esistenti. Fornisce capacità di reportistica essenziali per interpretazione e presentazione dei risultati dell'analisi.





| C ReportGenerator                                                                                             |
|---------------------------------------------------------------------------------------------------------------|
| input_path : str<br>output_path : str                                                                         |
| menu()<br>project_report(df)<br>run()<br>smell_report(df)<br>summary_report(df)<br>visualize_smell_report(df) |

### 2.2.7 utils

**Scopo:** Fornisce utility trasversali per operazioni di file I/O condivise tra i componenti del sistema.

#### Modulo principale:

- **file\_utils.py:** Classe statica con funzioni helper per gestione filesystem. Include metodi per pulizia directory, enumerazione ricorsiva di file Python, merge di risultati da multipli CSV, gestione logging thread-safe per esecuzioni parallele. Utilizzato da ProjectAnalyzer per operazioni batch e cleanup.

**Integrazione:** Package di supporto utilizzato trasversalmente da ProjectAnalyzer, Inspector e altri componenti per operazioni comuni su filesystem.

### 2.2.8 data\_preparation

**Scopo:** Pipeline completa per generazione di dataset sintetici di training tramite injection guidata da LLM di code smell ML-specific su funzioni estratte da repository GitHub.

#### Moduli principali:

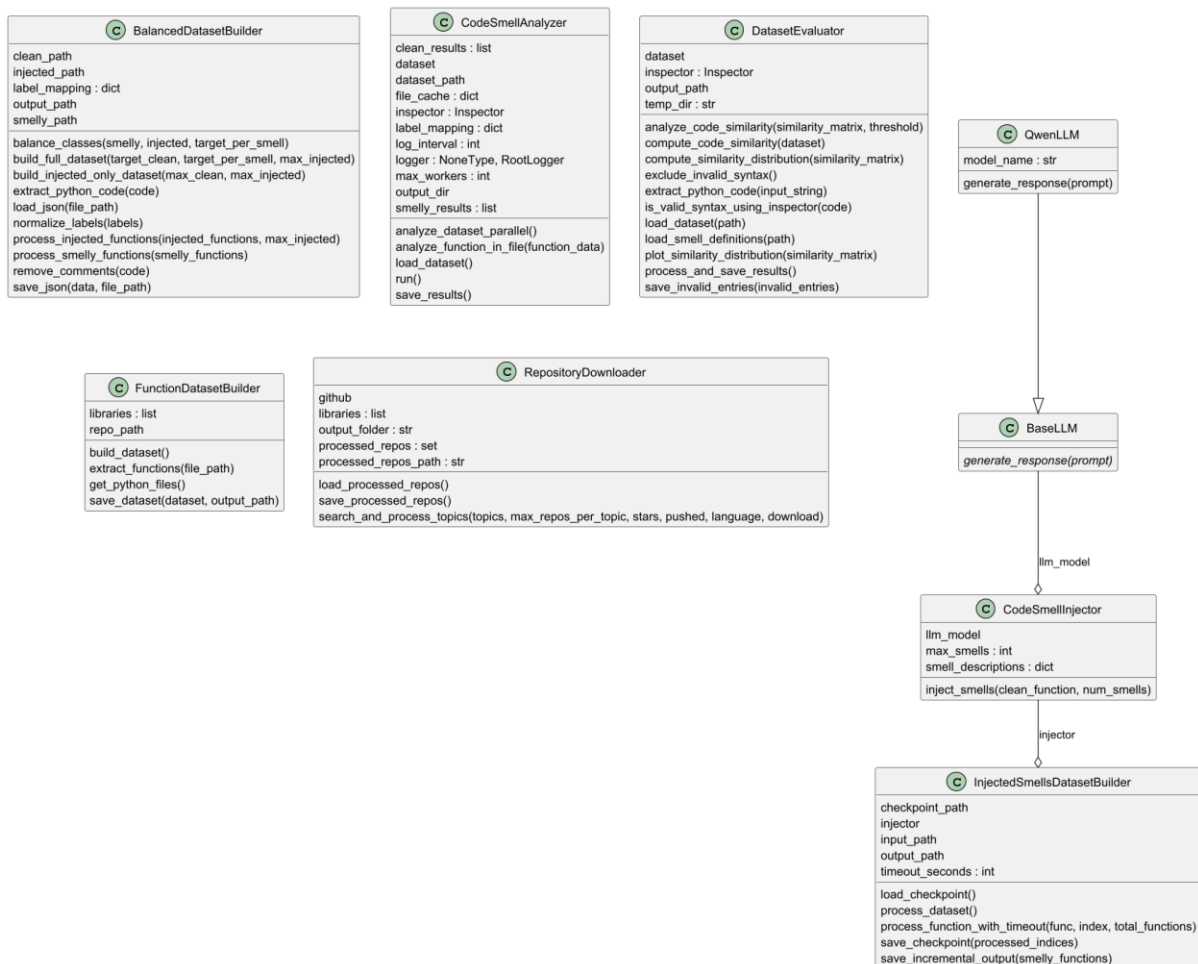
- **dataset\_creation\_runner.py:** Classe DatasetCreationRunner che orchestra la pipeline in 4 step sequenziali: (1) clonazione repository GitHub progetti ML, (2) estrazione funzioni ML-related, (3) analisi statica con CodeSmellAnalyzer, (4) injection smell sintetici.
- **function\_dataset\_builder.py:** Estrae funzioni Python da repository tramite AST. Filtra file ML-related basandosi su import (pandas, numpy, torch, tensorflow, sklearn), identifica funzioni con keyword ML (fit, predict, transform, model, loss, optimizer), salva in JSON. Supporta esecuzione parallela.
- **code\_smell\_analyzer.py:** Applica un motore di analisi statica (Inspector + RuleChecker) su dataset funzioni. Esecuzione parallela configurabile, salva i risultati annotati per filtrare funzioni clean.





- **code\_smell\_injector.py**: Inietta artificialmente smell in funzioni clean usando LLM. Dizionario di 16 smell con descrizioni/esempi, prompt engineering per injection controllata, selezione casuale 1-N smell per funzione, validazione sintattica codice generato. Utilizza l'interfaccia BaseLLM (Qwen via Ollama).
- **injected\_smells\_dataset\_builder.py**: Costruisce un dataset finale training JSON con struttura: function\_data, smelly (boolean), smells (lista annotazioni). Bilancia il dataset clean/smelly per evitare class imbalance.
- **dataset\_evaluator.py**: Validazione qualitativa dataset. Calcola metriche di distribuzione di smell, analizza la lunghezza delle funzioni, verifica la consistenza delle annotazioni.
- **repository\_downloader.py, balanced\_dataset\_builder.py**: Moduli support per clonazione repository e bilanciamento dataset.
- **qwen\_llm.py, base\_llm.py**: Astrazione LLM provider per generazione di codice smelly tramite Qwen/Ollama.

**Integrazione:** Package utilizzato offline per creazione dataset di training, separato dal flusso di analisi runtime (CLI/GUI). I dataset generati vengono utilizzati dal package finetuning per l'addestramento di modelli.





## 2.2.9 finetuning

**Scopo:** Pipeline di fine-tuning e validazione di modelli linguistici specializzati nella detection di ML code smells tramite tecniche di Parameter-Efficient Fine-Tuning (PEFT).

### Sotto-package principali:

- **train:** gestisce l'addestramento del modello
  - `model_trainer.py`: classe `ModelTrainer` che configura il modello preaddestrato (Qwen 2.5 Coder 3B) con LoRA (Low-Rank Adaptation) per fine-tuning efficiente, applicando 4-bit quantization e chat template Qwen-2.5 al tokenizer
  - `dataset_handler.py`: classe `DatasetHandler` che prepara i dataset da JSON, esegue split train/validation, tokenizza le conversazioni e converte le etichette multi-smell in formato HuggingFace Dataset
  - `training_configuration.py`: classe `TrainingConfiguration` che definisce parametri LoRA (rank, target\_modules, lora\_alpha, dropout) e `TrainingArguments` (batch size, learning rate, epochs, gradient accumulation)
  - `training_runner.py`: script entry point che orchestra il training utilizzando `SFTTrainer` (Supervised Fine-Tuning) e salva checkpoint/adapter LoRA
- **validation:** gestisce l'inferenza del modello fine-tuned
  - `model_inference.py`: classe `ModelInference` che carica checkpoint LoRA, ottimizza il modello con `FastLanguageModel.for_inference()` e genera predizioni con parametri configurabili (temperature, max\_new\_tokens, min\_p) su CPU/CUDA

**Integrazione:** riceve dataset bilanciati da `data_preparation`, addestra il modello con LoRA (rank 16-32) e 4-bit quantization via `unsloth/bitsandbytes`, produce modello fine-tuned con performance ~94.46% accuracy e ~95% precision/recall/F1, utilizzabile dalla webapp per detection LLM-based.



## 2.2.10 webapp

**Scopo:** Applicazione web moderna con architettura a microservizi per analisi interattiva di code smells tramite interfaccia browser, orchestrata via Docker Compose.



**Architettura:** API Gateway FastAPI (porta 8000) che funge da reverse proxy verso 3 microservizi backend indipendenti (FastAPI) + frontend Next.js/React (porta 3000), con networking bridge per comunicazione interna.

### **Componente Gateway:**

- gateway/main.py: configura FastAPI con middleware CORS, instrada richieste verso microservizi (/api/detect\_smell\_ai → porta 8001, /api/detect\_smell\_static → porta 8002, /api/generate\_report → porta 8003) tramite httpx.AsyncClient per chiamate asincrone, gestisce timeout ed errori di comunicazione

### **Microservizi Backend:**

- **AI Analysis Service** (porta 8001):
  - main.py: istanzia FastAPI app e registra router
  - routers/detect\_smell.py: endpoint POST /detect\_smell\_ai che valida input con AST, riceve {code\_snippet}, invoca Model.detect\_code\_smell() e restituisce DetectSmellResponse con logging errori
  - utils/model.py: client verso Ollama API (http://localhost:11434/api/generate), usa modello codesmile:latest (Qwen fine-tuned convertito in .gguf), parsing streaming response con accumulo chunk JSON, metodo parse\_smell() per estrazione strutturata
  - schemas/: Pydantic models (DetectSmellRequest, DetectSmellResponse, Smell) per request/response validation
- **Static Analysis Service** (porta 8002):
  - main.py: istanzia FastAPI app
  - routers/detect\_smell.py: endpoint POST /detect\_smell\_static
  - utils/static\_analysis.py: funzione detect\_static() che crea file temporaneo dallo snippet, invoca Inspector del core engine, converte DataFrame risultati in lista Smell, gestisce cleanup ed errori
  - schemas/: Pydantic models per validation
- **Report Service** (porta 8003):
  - main.py: istanzia FastAPI app
  - routers/report.py: endpoint POST /generate\_report che aggrega progetti analizzati con smell, genera report Excel/CSV con statistiche aggregate, restituisce file o summary JSON
  - utils/report\_generator.py: logica aggregazione risultati, costruzione DataFrame Pandas da smells, conteggi aggregati

### **Frontend Next.js:**

- **Package app:** layout globale e pagine (page.tsx con card navigation verso sezioni Analyze/Upload/Reports/About, upload-python/page.tsx per analisi singolo file)



con toggle AI/Static, file input con validazione .py, progress bar animata Framer Motion, display risultati con card per smell, gestione errori e toast notifications; upload-project/page.tsx per analisi multi-file con aggregazione risultati in tabella interattiva)

- **Package components:** componenti React riutilizzabili (HeaderComponent.tsx navbar con logo e navigation, FooterComponent.tsx con link e copyright, buttons, cards, modals, grafici, widget UI)
- **Package context:** contesto React per memorizzare lista progetti, aggiornare dati e condividere handler tra pagine
- **Package utils:** api.ts wrapper client TypeScript con funzioni detectAi(codeSnippet), detectStatic(codeSnippet), generateReport(projects) e error handling centralizzato via Axios interceptors
- **Package types:** tipizzazioni TypeScript condivise (smell, risposte API, contesto React)

**Deployment:** docker-compose.yml orchestra 5 container con variabili env (OLLAMA\_API\_URL per AI service, NEXT\_PUBLIC\_API\_BASE\_URL per webapp), volume mounts per codice core su Static service, networking bridge per comunicazione interna

#### Testing:

- cypress/: test E2E (upload-python.cy.ts test workflow upload singolo file, upload-project.cy.ts test workflow upload progetto, fixtures con sample code smells)
- \_tests\_/: test unit Jest per componenti React (rendering tests, interaction tests con React Testing Library, mock API calls)

**Integrazione:** frontend comunica con Gateway che instrada ai servizi specializzati; AI service usa modello fine-tuned da package finetuning; Static service riutilizza Inspector da package components; Report service aggrega risultati per visualizzazione.



## 3 Funzionamento e flussi di esecuzione

---

CodeSmile riconosce che gli sviluppatori lavorano in contesti molto diversi tra loro. Alcuni operano in ambienti altamente automatizzati; altri preferiscono un'esperienza più interattiva durante lo sviluppo; altri ancora lavorano in ambienti enterprise dove la centralizzazione e la collaborazione sono fondamentali. CodeSmile si adatta a tutte queste realtà offrendo tre punti di ingresso distinti.

### 3.1 Utilizzo tramite Linea di Comando (CLI)

La riga di comando rappresenta il modo più naturale per integrare CodeSmile in processi automatizzati. In questa modalità, lo sviluppatore comunica con CodeSmile specificando semplicemente dove trovare il codice e dove salvare i risultati. Il processo è completamente trasparente: l'utente non vede un'interfaccia, ma solo lo scorrimento dei log che informano sui progressi dell'analisi.

È particolarmente potente quando il volume di codice da analizzare è significativo. CodeSmile può infatti distribuire il lavoro su più processi paralleli, analizzando diverse parti del progetto contemporaneamente. Se per qualche ragione l'analisi viene interrotta, il sistema è sufficientemente intelligente da ricordare dove si era fermato e continuare da quel punto, evitando di ripetere il lavoro già svolto.

Questa modalità gestisce elegantemente anche più progetti contemporaneamente, orchestrando l'analisi multipla senza richiedere alcuna configurazione complessa.

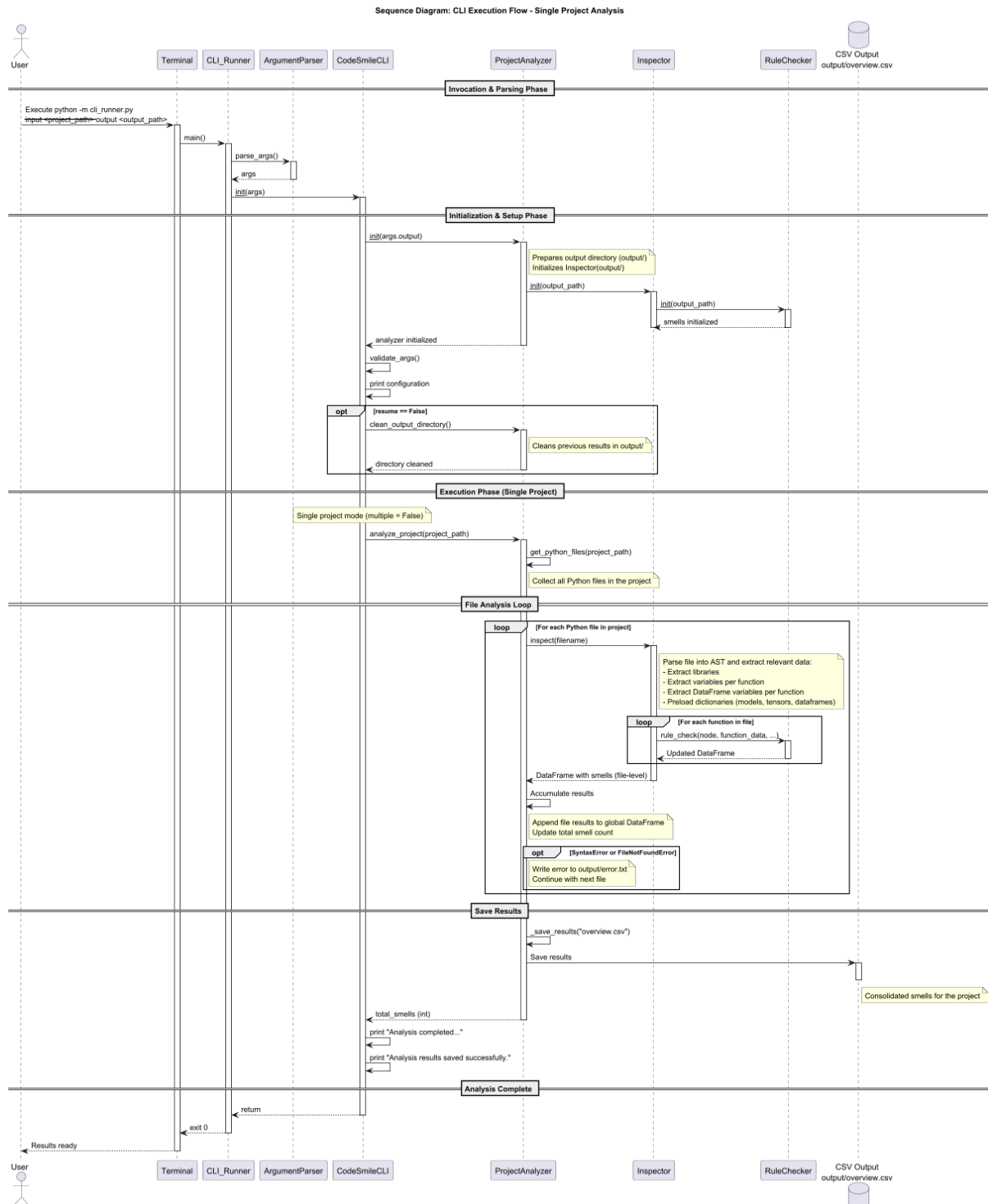
#### 3.1.1 Flusso di Utilizzo CLI

Il diagramma rappresenta uno scenario di esecuzione reale della modalità CLI su un singolo progetto, mantenendo un livello di dettaglio sufficiente a chiarire le interazioni principali senza riprodurre l'implementazione interna.

1. L'esecuzione ha inizio con l'invocazione del tool da terminale. Gli argomenti forniti dall'utente vengono analizzati e validati per determinare i parametri di esecuzione. Una volta completato il parsing, la CLI inizializza il flusso di analisi e stampa a console la configurazione selezionata, consentendo all'utente di verificare il contesto di esecuzione prima dell'avvio dell'analisi vera e propria.
2. Prima di procedere con l'analisi, il sistema prepara l'ambiente di output. Se l'esecuzione non è in modalità *resume*, eventuali risultati precedenti vengono rimossi e viene creata una nuova directory di output. Contestualmente, vengono inizializzati i componenti responsabili dell'analisi statica, in particolare l'analizzatore di progetto e il modulo di ispezione dei file Python.
3. In modalità *single project*, il sistema avvia l'analisi del progetto specificato in input. L'analizzatore individua ricorsivamente tutti i file Python presenti nella directory del progetto e li processa uno alla volta.
4. Ogni file Python viene trasformato in un Abstract Syntax Tree (AST), a partire dal quale vengono estratte le informazioni rilevanti per l'analisi. Su ciascuna funzione individuata vengono applicate le regole di rilevamento dei *code smell*, sia generiche sia specifiche per contesti AI/ML. I risultati ottenuti vengono progressivamente aggregati, mantenendo traccia del numero complessivo di anomalie rilevate.



5. Il flusso di esecuzione è progettato per essere robusto: eventuali errori di parsing o di accesso ai file non interrompono l'analisi complessiva. Tali eventi vengono registrati in un file di log dedicato e l'analisi prosegue con i file successivi.
6. Al termine dell'analisi, i risultati aggregati vengono salvati in un file CSV nella directory di output. Infine, la CLI notifica l'utente del completamento dell'analisi e del percorso in cui sono stati salvati i risultati.





## 3.2 Utilizzo tramite Interfaccia Utente Grafica (GUI)

La modalità GUI fornisce un'interfaccia desktop tradizionale e intuitiva, costruita con Tkinter, che rende l'analisi del codice accessibile a chi preferisce un'esperienza visuale rispetto a quella a riga di comando.

L'interfaccia presenta un layout semplice e intuitivo: bottoni per selezionare le cartelle da analizzare, piccoli controlli per configurare il comportamento dell'analisi, e un'ampia area di testo che mostra in tempo reale cosa sta succedendo mentre l'analisi procede.

L'esperienza è profondamente diversa dalla CLI proprio perché vi è un feedback visivo immediato e una sensazione di controllo più diretta. Mentre l'analisi procede, gli utenti vedono scorrere i log che informano sui file processati, gli errori trovati, il progresso complessivo. Questa retroazione in tempo reale è psicologicamente gratificante e permette di identificare rapidamente se qualcosa non sta andando come previsto.

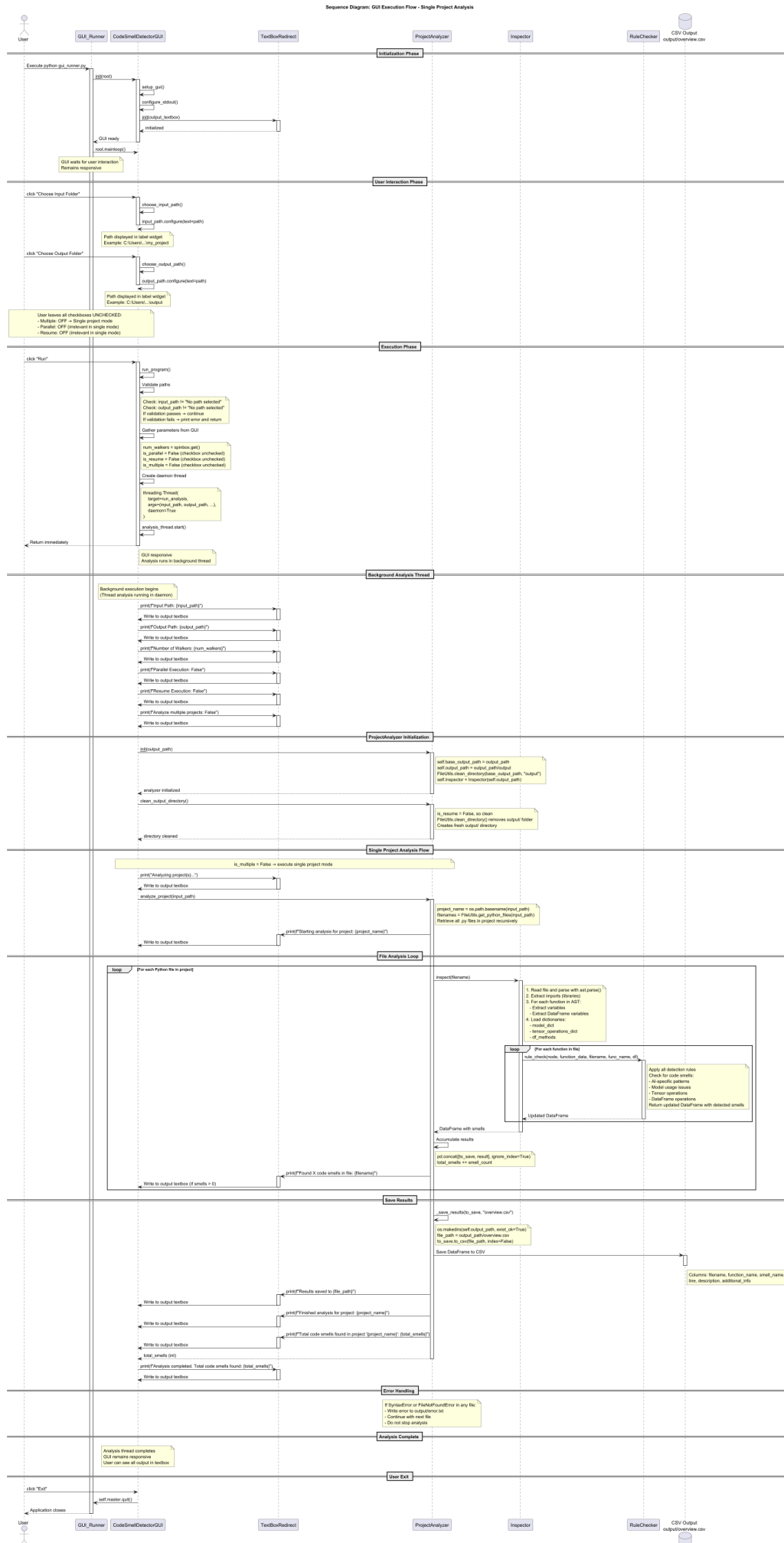
### 3.2.1 Flusso di Utilizzo GUI

Il diagramma di sequenza rappresenta uno scenario di esecuzione reale della modalità GUI di CodeSmile, riferito all'analisi di un singolo progetto Python.

Lo scenario considera il caso in cui l'utente avvii l'analisi senza abilitare la modalità multi-project, l'esecuzione parallela o il resume.

1. L'esecuzione ha inizio con l'avvio dell'applicazione GUI. Durante la fase di inizializzazione vengono creati i componenti grafici principali e configurato il sistema di output, che consente di visualizzare in tempo reale i messaggi prodotti durante l'analisi.
2. L'utente seleziona la directory di input contenente il progetto da analizzare e la directory di output in cui salvare i risultati. I parametri di configurazione vengono impostati tramite l'interfaccia grafica; lasciando disattivata l'opzione *Multiple*, l'analisi viene eseguita in modalità *single project*.
3. Al click sul pulsante di avvio, l'analisi viene eseguita in un thread separato, in modo da mantenere l'interfaccia grafica responsiva. I parametri selezionati vengono stampati nell'area di output della GUI per fornire un immediato riscontro all'utente.
4. Prima dell'analisi vera e propria, il sistema prepara l'ambiente di output. In assenza della modalità *resume*, eventuali risultati precedenti vengono rimossi, garantendo che l'esecuzione corrente produca risultati coerenti e indipendenti.
5. Il sistema individua ricorsivamente tutti i file Python presenti nel progetto. Ogni file viene analizzato singolarmente attraverso la costruzione dell'Abstract Syntax Tree (AST) e l'applicazione delle regole di rilevamento dei *code smell*. I risultati vengono progressivamente aggregati durante l'analisi.
6. Eventuali errori di parsing o di accesso ai file non interrompono l'esecuzione complessiva. Tali eventi vengono registrati e l'analisi prosegue con i file successivi, rendendo il flusso robusto rispetto a problemi locali.
7. Al termine dell'analisi, i risultati aggregati vengono salvati in un file CSV nella directory di output. La GUI notifica il completamento dell'operazione e mostra all'utente un riepilogo dell'esecuzione direttamente nell'area di output.









**AI-Specific Code Smells Detector**

Input Path: No path selected Choose Input Folder

Output Path: No path selected Choose Output Folder

Select number of walkers: 1

☐ Parallel ☐ Resume ☐ Multiple

Run Exit



**AI-Specific Code Smells Detector**

Input Path: C:/Users/aless/Desktop/IS/smell\_ai/input/projects/example Choose Input Folder

Output Path: C:/Users/aless/Desktop/IS/smell\_ai/output Choose Output Folder

Select number of walkers: 1

☐ Parallel ☐ Resume ☐ Multiple

Input Path: C:/Users/aless/Desktop/IS/smell\_ai/input/projects/example  
Output Path: C:/Users/aless/Desktop/IS/smell\_ai/output  
Number of Walkers: 1  
Parallel Execution: False  
Resume Execution: False  
Analyze multiple projects: False  
Starting analysis for project: example  
Found 20 code smells in file: C:/Users/aless/Desktop/IS/smell\_ai/input/projects/example/Code\_Smell\_Examples.py  
Results saved to C:/Users/aless/Desktop/IS/smell\_ai/output/output/overview.csv  
Finished analysis for project: example  
Total code smells found in project 'example': 20  
  
Analysis completed. Total code smells found: 20

Run Exit



## 3.2 Utilizzo tramite Web App

La WebApp rappresenta l'evoluzione moderna di CodeSmile, concepita per ambienti dove il codice non è una responsabilità individuale ma collettiva.

La WebApp funziona come una vera e propria piattaforma. Un utente accede tramite il browser a un'interfaccia moderna e reattiva, carica il codice che vuole analizzare, e il sistema si mette al lavoro. Ma piuttosto che fare tutto in un singolo processo monolitico, il lavoro viene distribuito a diversi servizi specializzati, ognuno responsabile di un aspetto specifico dell'analisi.

Un servizio si occupa dell'analisi statica basata su regole; un altro gestisce l'analisi con intelligenza artificiale e un terzo servizio genera report in PDF. Un coordinatore centrale (il gateway) gestisce il flusso di queste richieste, garantendo che tutto funzioni armoniosamente.

La WebApp inoltre garantisce che il codice può essere analizzato ovunque, non solo dal computer locale dello sviluppatore. Questo è fondamentale in ambienti distribuiti, dove i team sono sparsi geograficamente, o dove il codice risiede su server centralizzati.

### 3.3.1 Flusso di Utilizzo WebApp

Questo scenario descrive il flusso di utilizzo della WebApp di CodeSmile per l'esecuzione dell'analisi statica di codice Python.

1. L'utente visita la pagina "Upload and Analyze Python Code", dove vede il selettore della modalità, il campo per scegliere un file .py e il pulsante di avvio.
2. L'utente clicca sul selettore file e sceglie un file con estensione .py. Il nome del file selezionato viene mostrato nell'interfaccia.
3. L'utente imposta la modalità di analisi su "Static Tool".
4. L'utente clicca su "Upload Code (Static Mode)". L'app verifica che il file esista e che non sia vuoto; se manca qualcosa, mostra un messaggio di errore.
5. Il contenuto del file viene letto direttamente dal browser (come testo) e preparato per l'invio al backend.
6. Il frontend invia una richiesta POST al servizio di analisi statica contenente lo snippet di codice (code\_snippet) in formato JSON.
7. Il backend crea un file temporaneo .py, esegue l'analisi tramite Inspector e applica le regole con RuleChecker per individuare eventuali ML-specific code smells.
8. Quando l'analisi termina, l'app aggiorna la pagina e mostra:
  - a. una lista di risultati (card con smell, descrizione, funzione e linea) se sono stati trovati smell;
  - b. oppure un messaggio informativo se non sono stati rilevati smell.

